

CS 4530: Fundamentals of Software Engineering

Module 08: Concurrency Patterns in Typescript

Adeel Bhutta and Mitch Wand
Khoury College of Computer Sciences

Learning Goals for this Lesson

- At the end of this lesson, you should be prepared to:
 - Explain the difference between JS run-to-completion semantics and interrupt-based semantics.
 - Given a simple program using `async/await`, work out the possible orders in which the statements in the program will run.
 - Write simple programs that create and manage promises using `async/await`
 - Write simple programs to mask latency with concurrency by using non-blocking IO and `Promise.all` in TypeScript.

Why async matters

- Your browser probably spends 99% of its time waiting
 - for http requests to return
 - for database calls to return
- Wouldn't it be nice if we could use that waiting time for something else?
 - maybe we have some other http request we could launch
 - maybe other things (throbbers??)

Typescript/Javascript achieves this goal using two techniques:

1. non-blocking IO

2. cooperative multiprocessing

Part 1: JS/TS in your browser has some primitives for doing non-blocking I/O

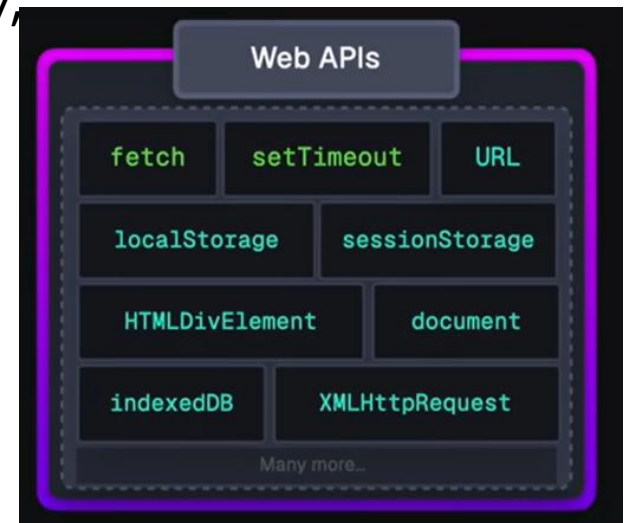
- This is called the "Web API"
- These are things like http requests, I/O operations, or timers.
- Each of these returns a promise that you can **await**. The promise runs while it is pending, and produces the response from the http request, or the contents of the file, etc.
- You may use some of these directly, like **fetch** or **setTimeout**.
- More frequently, you will invoke these using a convenient library, like

```
axios.get('https://rest-example.covey.town')
```

to make an http request, or

```
fs.readFile(filename)
```

to read the contents of a file.



Part 2: Javascript/Typescript uses **cooperative multiprocessing**

- In cooperative multiprocessing, **only one process is executed at a time.**
- **Each process pauses when it is convenient** to allow other processes to make progress.
- To make this practical (and avoid cheating!), we need a programming model that encourages this behavior

async/await is a programming model for cooperative multiprocessing

- In async/await, the program is organized into a set of "async functions"
- An async function always returns a promise.
- If you **await** on a promise, your program pauses its execution until async function is completed. When one program pauses, the runtime can choose to resume executing any process that is ready to run.

Quick Review of Promises (from M04)

- Typescript uses the type system to distinguish long-running computations
- A long-running (IO-bound) computation gets type **Promise<DataRecord>**, rather than **DataRecord**.
- **async** functions return Promises
- Use **await** to get the result of a promise
- Every occurrence of **await** must be in an async function
- A **Promise<void>** is executed for effect, so it doesn't make sense to **await** its result.

A promise can be in one of exactly 3 states

- A JavaScript promise can be in one of three states: **pending**, **fulfilled**, or **rejected**.
- **Pending** is the initial state where the promise is ***waiting*** for an operation to complete; Then it becomes **Ready** for Execution; and eventually it is **Executing** before being resolved
- **Resolved**: either fulfilled or rejected.
 - **fulfilled** means the operation was successful,
 - **rejected** indicates that the operation failed.

Async functions return Promises

```
async function someFunction(i: number) : Promise<number>
{
    const j = i + 1;
    // ...
    const k = await someOtherAsyncFunction(j);
    // ...
    const m = k + 100;
    return m;
}
```

This function returns a promise that will resolve with **m**.

```
async function someFunction(i: number) : Promise<number>
{
    const j = i + 1;
    // ...
    const k = await someOtherAsyncFunction(j);
    // ...
    const m = k + 100
    return m;
}
```

Run to Completion is a central concept

- In JS, you work on one and one task only (i.e., event loop has a single call stack)
- Also, if you start a task, you must finish it (i.e., call stack must be emptied) before you start another task. This pattern is called "run-to-completion" semantics
- A task can be paused in exactly two places:
 - when it reaches an await
 - when it reaches a return

This function can only be paused in certain places

```
async function someFunction(i: number) : Promise<number> {
```

```
  const j = i + 1;  
  // ...
```

```
  const k = await someOtherAsyncFunction(j);
```

```
  // ...  
  const m = k + 100;  
  return m;
```

```
}
```

It will never pause in here. This is called a Critical Section

Or in here (this is another Critical Section)

What happens at those pause points?

```
async function someFunction(i: number) {
```

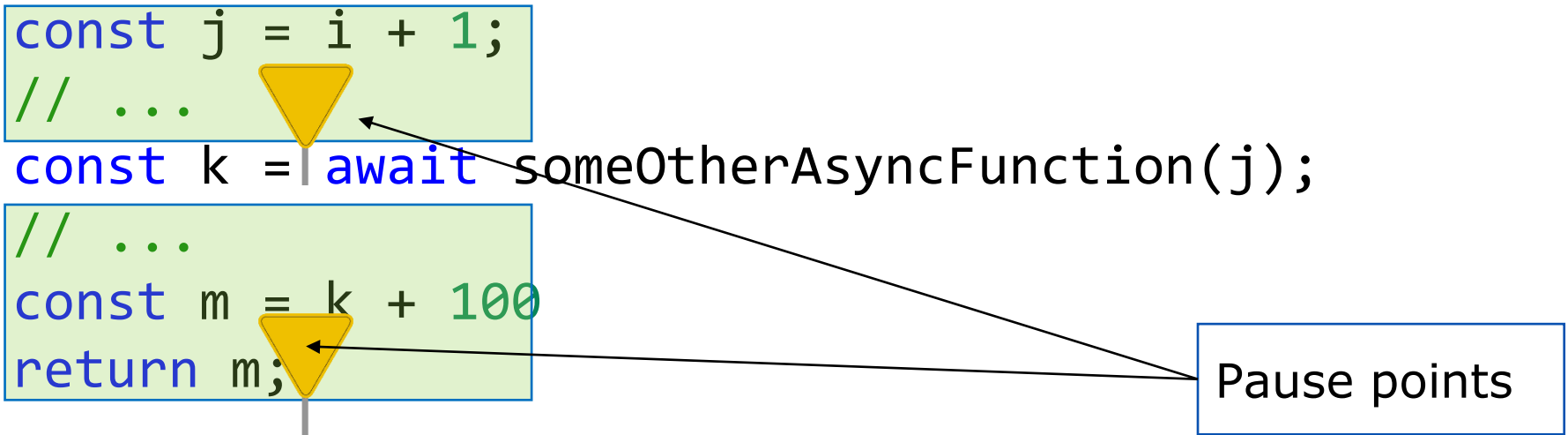
```
  const j = i + 1;  
  // ...
```

```
  const k = await someOtherAsyncFunction(j);
```

```
  // ...  
  const m = k + 100;  
  return m;
```

```
}
```

Pause points

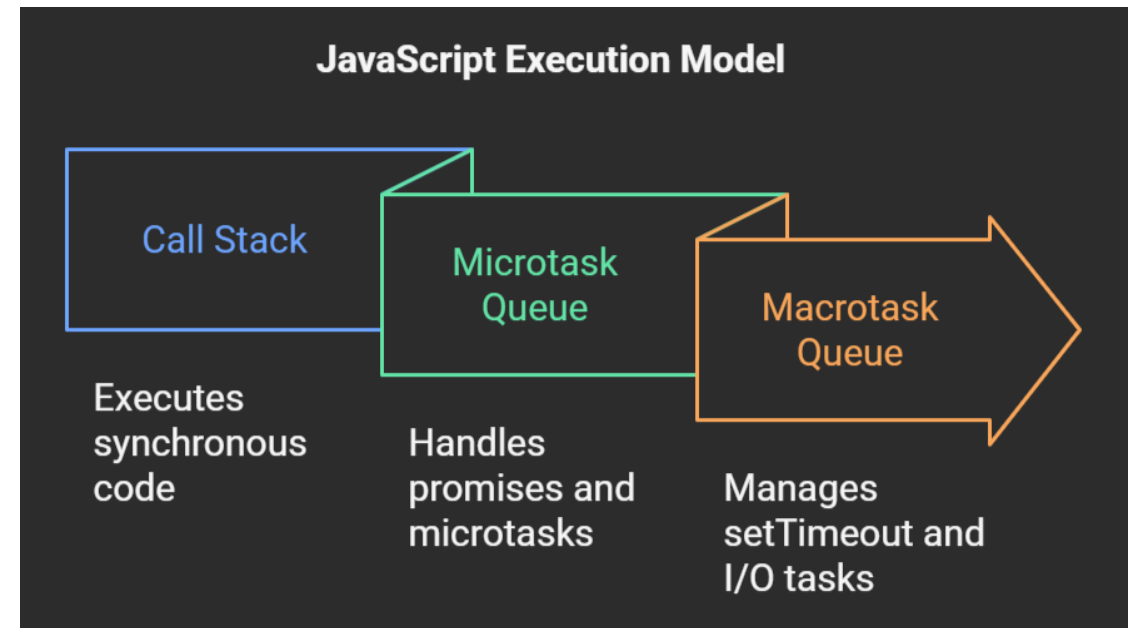


What happens when you await?

- When you 'await' on an asynchronous task, you are cooperating with the event loop by allowing it to work on something else.
- In technical terms, you are **returning** with a ***pending*** promise which means that some work will be completed in future.
- While you are waiting for that future, event loop will continue finishing its current task (whatever is in call stack) and then other tasks (whatever is in callback queue).
- Eventually you will return to complete the ***awaited*** work.

Promises come in two flavors: microtasks and macrotasks

- Microtask queue is used for immediate/urgent tasks while Macrotask queue is used for slower asynchronous tasks.
- When call stack is empty, microtask queue is processed first (before browser renders). When microtask queue is empty then macrotask queue is processed.



Macrotasks are API calls; Microtasks are what you do after the API call returns

```
async function someFunction(i: number) {
```

```
  const j = i + 1;  
  // ...
```

```
  const k = await someOtherAsyncFunction(j);
```

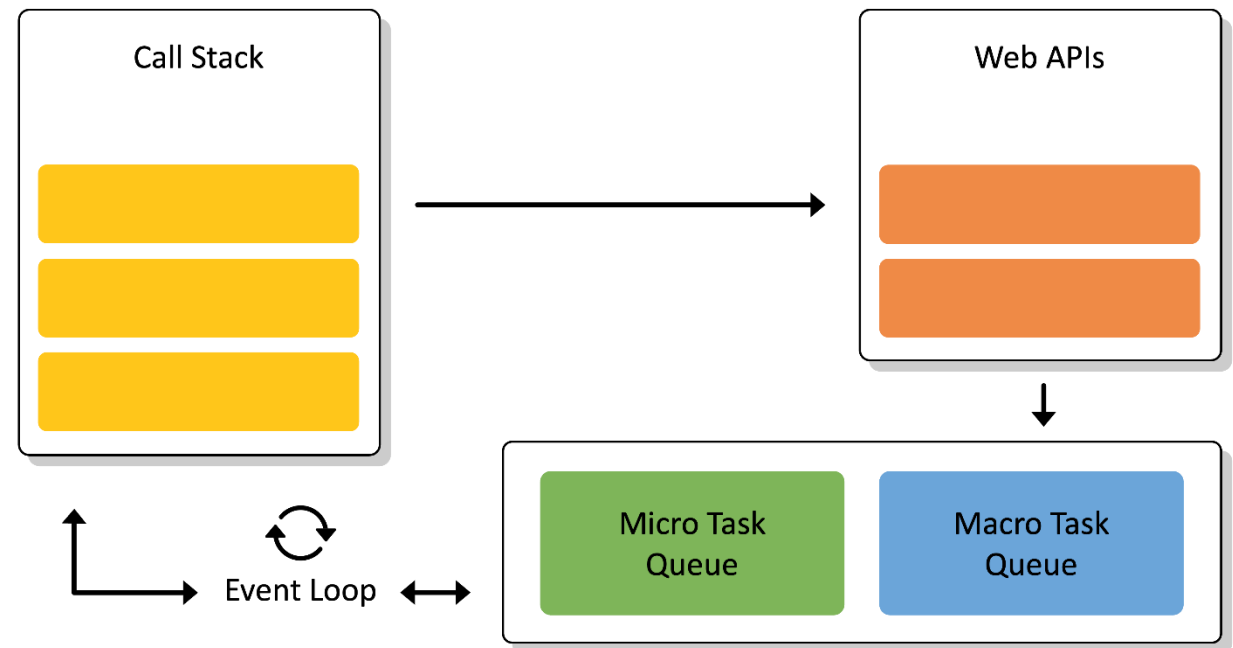
```
  // ...  
  const m = k + 100;  
  return m;  
}
```

This will be
a macrotask

This will be a
microtask

How does JS Event Loop make this happen?

- One Event Loop means that we have single thread of execution
- The WebAPI is used for asynchronous tasks
- Queues are used for "await"-ed tasks
- When call stack becomes **empty**, the event loop picks up tasks from Callback Queues
- Microtask queue has priority over macrotask queue



By contrast, most OS's use **pre-emptive multiprocessing**

- OS manages multiprocessing with multiple threads of execution
- Processes may be ***interrupted*** at unpredictable times
- Inter-process communication by shared memory
- Data races abound
- Really, really hard to get right: need critical sections, semaphores, monitors (all that stuff you learned about in op. sys.)

Java vs. JS/TS

```
async function asyncPrintA() {  
    await waitRandom(1);  
    console.log('A');  
}
```

```
async function asyncPrintBC() {  
    await waitRandom(2);  
    console.log('B');  
    console.log('C');  
}
```

```
async function run() {  
    await Promise.all([asyncPrintA(), asyncPrintBC()])  
}
```

```
run()
```

- In Java, you could get an interrupt between the print B and the print C.
- So the output could be ABC, BAC, BCA
- In TS/JS, the print B and print C are in the same critical section, so BAC is impossible!

A simple request

```
// fakeRequest(n) is an async that waits for 1 second and then
// resolves with the number n+10
import { fakeRequest } from "./fakeRequest";
import { timeIt } from "./timeIt";

async function main() {
  console.log('main started');
  const res = await fakeRequest(32);
  console.log(`fakeRequest(${request}) returned: ${res}`);
  console.log('main done');
}

timeIt(main)
```

```
export function fakeRequest(n: number): Promise<number> {
  return new Promise((resolve, reject) => {
    console.log('fakeRequest received request:', n);
    setTimeout(() => {
      console.log('time passes....');
      resolve(n + 10);
    }, 1000);
  });
}
```

```
$ node oneRequest.ts
main started
fakeRequest received request: 32
time passes....
fakeRequest(32) returned: 42
main done
1015.98 msec
```

Pattern for starting a concurrent computation using non-blocking I/O

```
export async function makeRequest(requestNumber:number) {  
  console.log(`starting makeRequest(${requestNumber})`);  
  const response = await fetch('https://rest-example.covey.town');  
  console.log('request:', requestNumber, '; response:', response.data);  
}
```

1. The first console.log is printed
2. The http request is *immediately* sent, using non-blocking i/o.
3. A pending promise is returned to the caller. Caller can continue its other business.
4. Eventually, the fetch returns.
5. Some time after that, the console.log is printed and the makeRequest concludes.
6. Any promises that are waiting for the result of this makeRequest become eligible for execution.

Three Requests, sequentially

```
async function main() {  
  console.log('starting main');  
  const res1 = await fakeRequest(1);  
  console.log(`fakeRequest(1) returned: ${res1}`);  
  const res2 = await fakeRequest(2);  
  console.log(`fakeRequest(2) returned: ${res2}`);  
  const res3 = await fakeRequest(2);  
  console.log(`fakeRequest(2) returned: ${res3}`);  
  console.log('main done');  
}  
  
timeIt(main)
```

```
$ node threeRequestsSequentially.ts  
starting main  
fakeRequest received request: 1  
time passes....  
fakeRequest(1) returned: 11  
fakeRequest received request: 2  
time passes....  
fakeRequest(2) returned: 12  
fakeRequest received request: 3  
time passes....  
fakeRequest(3) returned: 13  
main done  
3024.03 msec
```

src/Slides/threeRequestsSequentially.ts

Use Promise.all to execute several requests concurrently

- Promise.all takes a list of Promise<T> and creates a Promise<T[]>
 - ie, a promise that resolves to a list of T's
- When executed, this promise runs them *concurrently*,
- and resolves with the list of values
 - (assuming they all resolve successfully)

Use Promise.all to execute several requests concurrently

src/Slides/threeRequestsConcurrently.ts

```
async function main() {  
  console.log('starting main');  
  const promises = [fakeRequest(1),  
                    fakeRequest(2),  
                    fakeRequest(3)]  
  const results = await Promise.all(promises)  
  console.log('results:', results);  
  console.log('main done');  
}
```

timeIt(main)

- 3 requests may finish in any order

```
$ node threeRequestsConcurrently.ts  
starting main  
fakeRequest received request: 1  
fakeRequest received request: 2  
fakeRequest received request: 3  
time passes....  
time passes....  
time passes....  
results: [ 11, 12, 13 ]  
main done  
1018.81 msecno
```

Why is that?

Visualizing Promise.all

Sequential (await)

“Don’t make another request until you got the last response back”

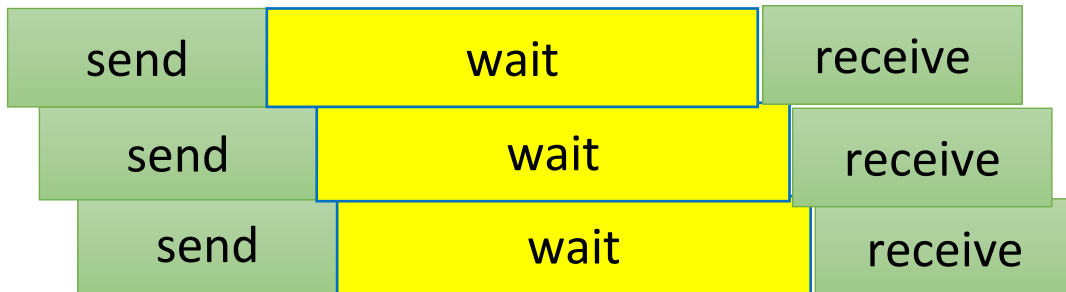
237 msec



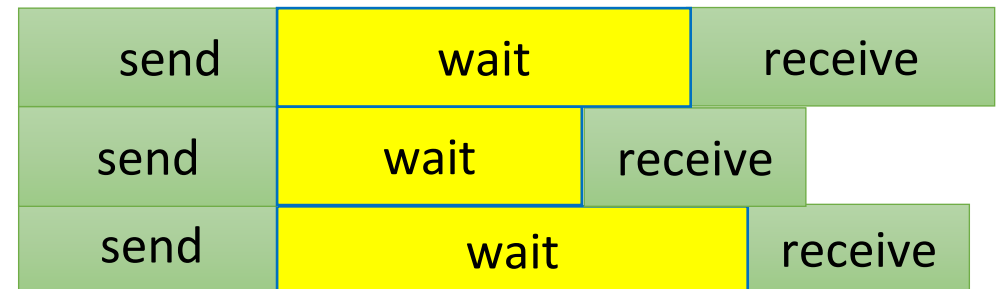
Concurrent (Promise.all)

“Make all of the requests now, then wait for all of the responses”

34 msec



or

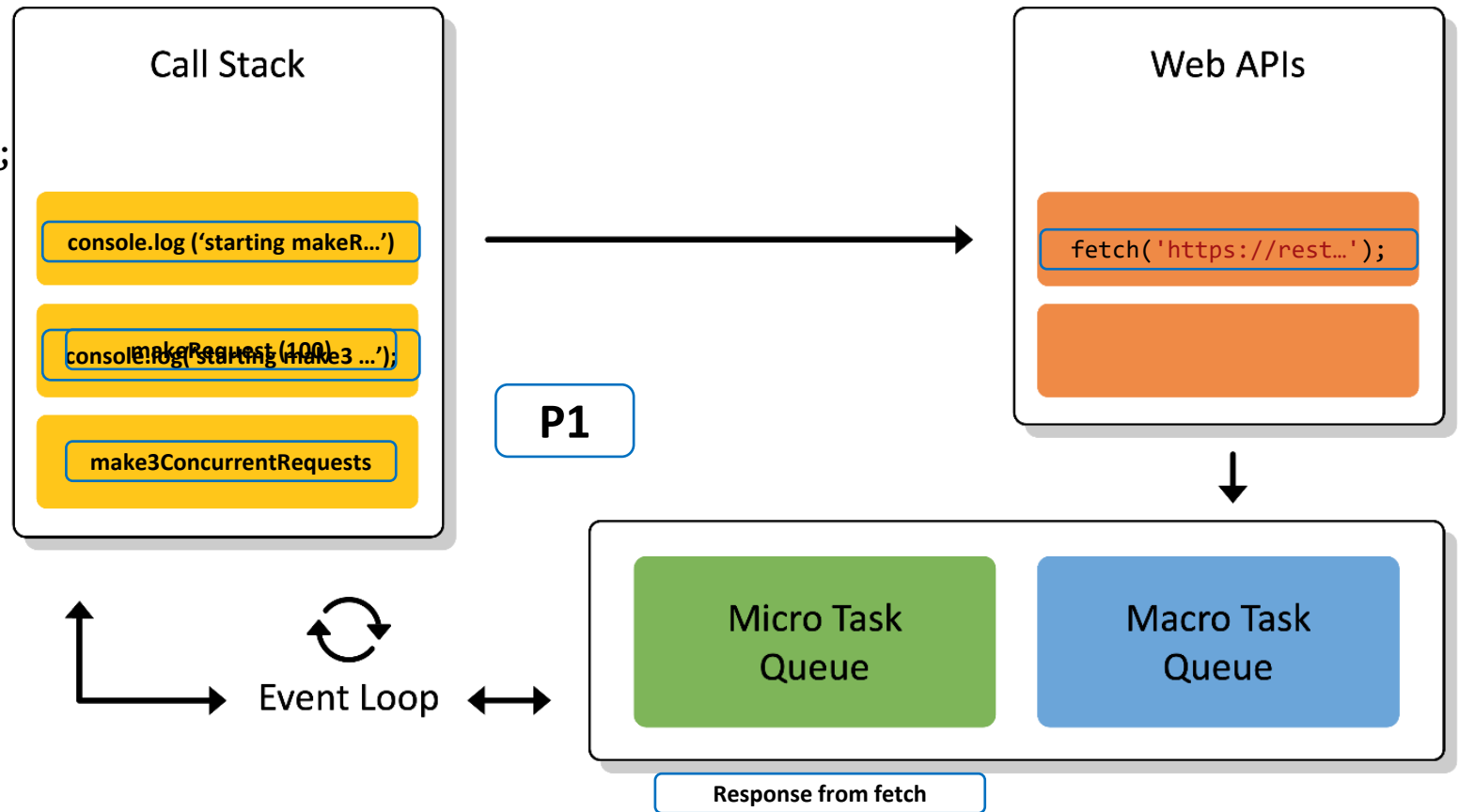


Running 3 concurrent fetch requests (start)

```
async function main() {  
  console.log('starting main');  
  const promises = [fakeRequest(1),  
                    fakeRequest(2),  
                    fakeRequest(3)];  
  const results = await  
    Promise.all(promises);  
  console.log('results:', results);  
  console.log('main done');  
}
```

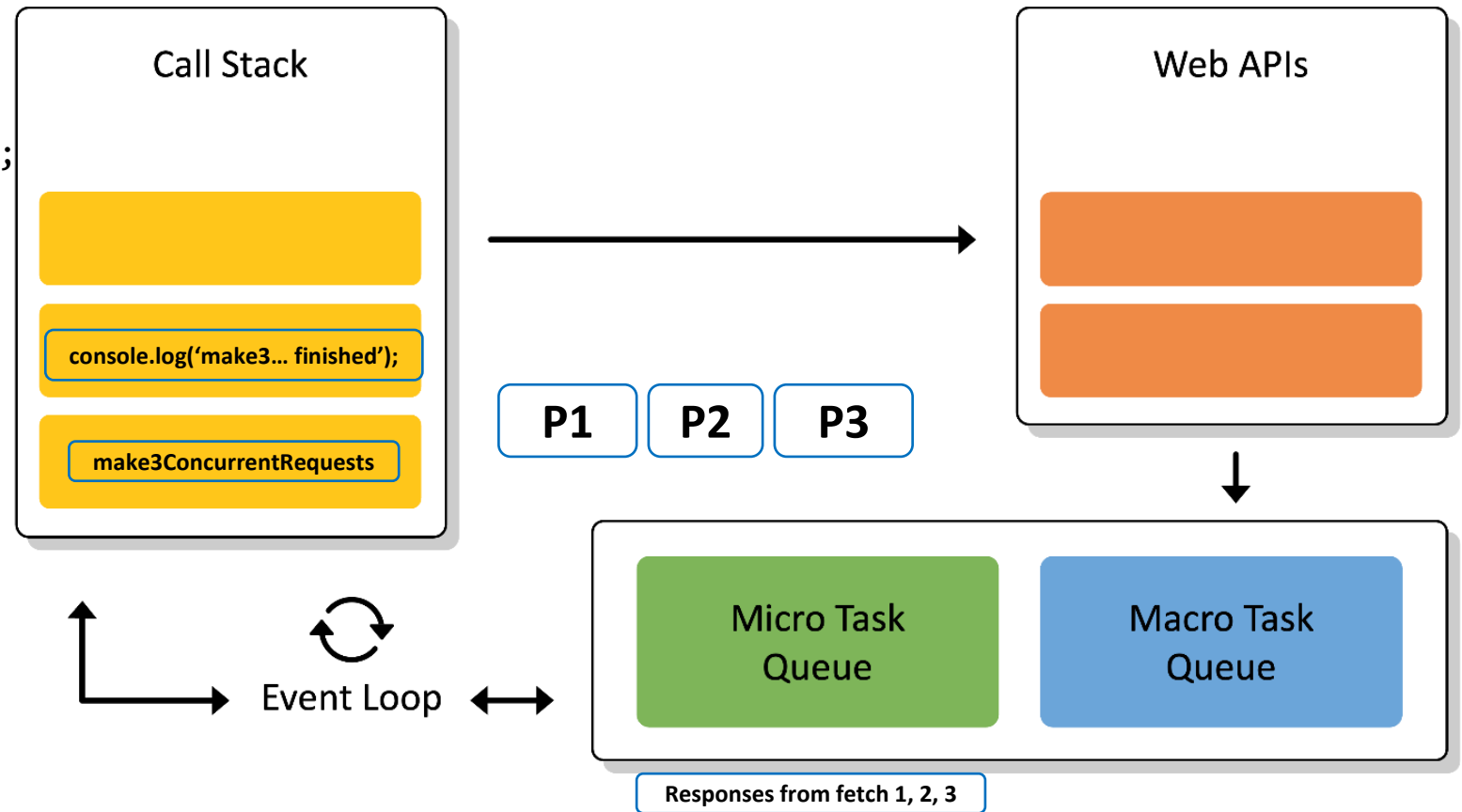
timeIt(main)

- Now, makeRequest(200) can proceed ...



Running 3 concurrent fetch requests (end)

```
async function main() {  
  console.log('starting main');  
  const promises = [fakeRequest(1),  
                    fakeRequest(2),  
                    fakeRequest(3)];  
  const results = await  
    Promise.all(promises);  
  console.log('results:', results);  
  console.log('main done');  
}  
  
timeIt(main)
```



Recover from errors with try/catch

```
// a request that may fail
async function maybeFailingRequest(req: number): Promise<number> {
  const res = await fakeRequest(req);
  if (res < 0) {
    throw new Error(`Request ${req} failed because response ${res} < 0`);
  } else {
    return res;
  }
}
```

src/Slides/tryCatchExample.ts

try/catch, continued

```
async function main() {  
  console.log('main started');  
  const req1 = -32  
  let res: number;  
  try {  
    res = await maybeFailingRequest(req1);  
    console.log(`fakeRequest(${req1}) returned: ${res}`);  
  } catch (err) {  
    console.error(`Error occurred for request ${req1}`);  
    res = 0  
  }  
  console.log('main done with res =',  
}
```

```
timeIt(main);
```

```
$ node tryCatchExample.ts  
main started  
fakeRequest received request: -32  
time passes....  
Error occurred for request -32  
main done with res = 0  
1007.52 msec
```

Pattern for testing an async function

```
import { fakeRequest } from './fakeRequest.ts'
import { describe, expect, test } from 'vitest'
```

src/Slides/example.spec.ts

```
// fakeRequest should make a network request using myFetch
// and then returns n + 1
```

```
describe('fakeRequest', () => {

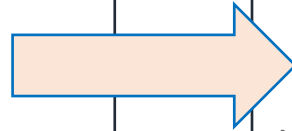
  test('fakeRequest should return its argument + 1 (await/expect)', async () => {
    expect.assertions(1)
    await expect(fakeRequest(100)).resolves.toEqual(101)
  })

  test('fakeRequest should return its argument + 1 (expect/await)', async () => {
    expect.assertions(1)
    expect(await fakeRequest(7)).toEqual(8)
  })
})
```

Other things you should know about async

Async/await code is compiled into promise/then code. Using await is preferred

```
async function
makeThreeSerialRequests() {
1.  console.log('Making first
request');
2.  await makeOneGetRequest();
3.  console.log('Making second
request');
4.  await makeOneGetRequest();
5.  console.log('Making third
request');
6.  await makeOneGetRequest();
7.  console.log('All done!');
}
makeThreeSerialRequests();
```



```
console.log('Making first request');
makeOneGetRequest().then( () => {
    console.log('Making second request');
    return makeOneGetRequest();
}).then( () => {
    console.log('Making third request');
    return makeOneGetRequest();
}).then( () => {
    console.log('All done!');
});
```

async doesn't eliminate data races

```
import { myFetch } from './fakeRequest.ts';

const log: string[] = [];

async function X(n: number) {
  await myFetch(1)
  log.push('X');
}

async function Y(n: number) {
  await myFetch(2)
  log.push('Y');
}

for (let i = 0; i < 4; i += 1) {
  log.push('start');
  await Promise.all([X(10), Y(20)]);
}

console.log(log);
```

```
$ node dataRace.ts
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
[
  'start', 'X', 'Y',
  'start', 'Y', 'X',
  'start', 'Y', 'X',
  'start', 'Y', 'X'
]
```

Review

- You should now be prepared to:
 - Explain the difference between JS run-to-completion semantics and interrupt-based semantics.
 - Given a simple program using `async/await`, work out the possible orders in which the statements in the program will run.
 - Write simple programs that create and manage promises using `async/await`
 - Write simple programs to mask latency with concurrency by using non-blocking IO and `Promise.all` in TypeScript.